Final Project Part 2

Maximilien Filion - 20598645,
Aneesh Kaura - 66832304,
Monica Mihailescu - 47409289

2025-11-17

==========

RNA-seq Analysis Template: Clustering and Differential Expression

==========

This template provides a structured framework for comprehensive gene expression analysis from preprocessing through clinical validation

==========

Load required libraries

```
#if (!requireNamespace("BiocManager", quietly = TRUE))
#     install.packages("BiocManager")
# BiocManager::install("org.Hs.eg.db")
# BiocManager::install("AnnotationDbi")

# Install clusterProfiler properly
#BiocManager::install("clusterProfiler", ask = FALSE, update = FALSE)

#BiocManager::install("DOSE")
#BiocManager::install("pathview")

library(clusterProfiler)

library(AnnotationDbi)

library(DESeq2)
```

```r
library(pheatmap)

library(ggplot2)

library(cluster)
library(survival)
library(survminer)

#library(clusterProfiler)
library(org.Hs.eg.db) # Contains human gene annotations

library(dplyr)

library(tibble)
library(biomaRt)
```

set.seed(1)

==========

STEP 1: DATA LOADING AND INITIAL SETUP

==========

Load the data files
- count_matrix: raw counts (genes x samples)
- clinical_data: patient clinical information
- mutation_data: mutation status for each patient

```r
#Load the data (from lab 7)
RNAseq_KIRC <- read.csv("RNAseq_KIRC.csv")
data_clinical_patient <- read.delim("data_clinical_patient.txt", header = TRU
E, sep = "\t", skip = 4)
data_mutations <- read.delim("data_mutations.txt", header = TRUE, sep = "\t",
stringsAsFactors = FALSE)

head(RNAseq_KIRC)
```

Creates a dataframe containing all the patients common to the three datasets.

will be used in downstream applications.

```r
# RNAseq_KIRC dataset
rna_patient_ids <- gsub("\\.", "-", substr(colnames(RNAseq_KIRC), 1, 12))
# Data_clinical_patient dataset
```

```r
clinical_patient_ids <- unique(data_clinical_patient$PATIENT_ID)
# Data_mutation datasetd
mutation_patient_ids <- unique(substr(data_mutations$Tumor_Sample_Barcode, 1,
12))
# Common patients
common_patients <- Reduce(intersect, list(rna_patient_ids,
clinical_patient_ids, mutation_patient_ids))

# Patients in all three datasets
common_patients <- Reduce(intersect, list(
  rna_patient_ids,
  clinical_patient_ids,
  mutation_patient_ids
))
cat("Number of patients in ALL datasets:", length(common_patients), "\n")

## Number of patients in ALL datasets: 354
```

Trim the datasets to only include the common patients

```r
#For the RNA seq dataset, we want to keep all the common patient columns and
the ensembl gene ID (i.e.Column "X")
# Trim the RNA sequencing dataset
common_patients_and_X <- c(common_patients, "X")
RNAseq_KIRC <- RNAseq_KIRC[ , gsub("\\.", "-", substr(colnames(RNAseq_KIRC),
1, 12)) %in% common_patients_and_X]

# Trim the clinical dataset
data_clinical_patient <- data_clinical_patient[data_clinical_patient$PATIENT_
ID %in% common_patients, ]

# Trim the mutation dataset
data_mutations <- data_mutations[substr(data_mutations$Tumor_Sample_Barcode,
1, 12) %in% common_patients, ]
```

The code above only cuts ~50 duplicate genes and takes a while to run. It was commented to signify that we thought about it but did not use it due to computational ressources.

Check for missing values

```r
# RNA-seq:
rnaseq_missing <- sum(is.na(RNAseq_KIRC))
cat("Missing values in RNA-seq count matrix:", rnaseq_missing, "\n")

## Missing values in RNA-seq count matrix: 0

# Clinical: Check key columns
key_clinical_cols <- c("PATIENT_ID", "AGE", "SEX", "OS_STATUS", "OS_MONTHS",
"DSS_STATUS", "DSS_MONTHS", "DFS_STATUS", "DFS_MONTHS", "PFS_STATUS", "PFS_MO
```

```
NTHS", "PATH_T_STAGE", "PATH_N_STAGE", "PATH_M_STAGE", "AJCC_PATHOLOGIC_TUMOR
_STAGE")
existing_cols <- key_clinical_cols[key_clinical_cols %in% colnames(data_clini
cal_patient)]

if(length(existing_cols) > 0) {
  for(col in existing_cols) {
    missing_count <- sum(is.na(data_clinical_patient[[col]]) | data_clinical_
patient[[col]] == "")
    cat("Missing/empty values in", col, ":", missing_count, "\n")
  }
}
```

```
## Missing/empty values in PATIENT_ID : 0
## Missing/empty values in AGE : 0
## Missing/empty values in SEX : 0
## Missing/empty values in OS_STATUS : 0
## Missing/empty values in OS_MONTHS : 0
## Missing/empty values in DSS_STATUS : 6
## Missing/empty values in DSS_MONTHS : 0
## Missing/empty values in DFS_STATUS : 255
## Missing/empty values in DFS_MONTHS : 255
## Missing/empty values in PFS_STATUS : 0
## Missing/empty values in PFS_MONTHS : 2
## Missing/empty values in PATH_T_STAGE : 0
## Missing/empty values in PATH_N_STAGE : 0
## Missing/empty values in PATH_M_STAGE : 2
## Missing/empty values in AJCC_PATHOLOGIC_TUMOR_STAGE : 0
```

```
# Mutations: Check key columns
cat("\nMissing values in Tumor_Sample_Barcode:", sum(is.na(data_mutations$Tum
or_Sample_Barcode)), "\n")
```

```
##
## Missing values in Tumor_Sample_Barcode: 0
```

```
cat("Missing/empty values in Hugo_Symbol:", sum(is.na(data_mutations$Hugo_Sym
bol) | data_mutations$Hugo_Symbol == ""), "\n")
```

```
## Missing/empty values in Hugo_Symbol: 0
```

```
cat("Missing/empty values in Variant_Classificatoin:", sum(is.na(data_mutatio
ns$Variant_Classification) | data_mutations$Variant_Classificationl == ""), "
\n")
```

```
## Missing/empty values in Variant_Classificatoin: 0
```

```
cat("Missing/empty values in Consequence:", sum(is.na(data_mutations$Conseque
nce) | data_mutations$Consequence == ""), "\n")
```

```
## Missing/empty values in Consequence: 0
```

```r
cat("Missing/epmty values in Impact:", sum(is.na(data_mutations$IMPACT) | dat
a_mutations$IMPACT == ""), "\n\n")
```

```
## Missing/epmty values in Impact: 0
```

==========

STEP 2: PREPROCESSING

==========

2.1: Normalization using DESeq2

```r
# Create DESeq2 object from raw counts
# Prepare count matrix (genes as rows, samples as columns)
count_matrix <- as.matrix(RNAseq_KIRC[, -1])  # Remove gene name column
rownames(count_matrix) <- RNAseq_KIRC[, 1]    # Set gene names as row names

# Create a simple sample metadata data frame
# For normalization, we just need a basic colData with sample names
coldata <- data.frame(
  sample = colnames(count_matrix),
  condition = rep("tumor", ncol(count_matrix))  # Placeholder condition
)
rownames(coldata) <- coldata$sample

cat("Count matrix dimensions:", dim(count_matrix), "\n")
```

```
## Count matrix dimensions: 60660 427
```

```r
# Create DESeq2 dataset object
dds <- DESeqDataSetFromMatrix(
  countData = count_matrix,
  colData = coldata,
  design = ~ 1  # No DE, normalization only
)
cat("DESeq2 object created successfully\n\n")
```

```
## DESeq2 object created successfully
```

```r
# Estimate size factors
dds <- estimateSizeFactors(dds)

# Get size factors
size_factors <- sizeFactors(dds)
cat("Size factor summary:\n")
```

```
## Size factor summary:

print(summary(size_factors))

##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.2023  0.8354  1.0476  1.0507  1.2303  2.3956

cat("\n")

# Extract normalized counts
normalized_counts <- counts(dds, normalized = TRUE)
cat("Normalized count matrix dimensions:", dim(normalized_counts), "\n")

## Normalized count matrix dimensions: 60660 427
```

2.2: Log-transformation

```
# simple Log2(x+1) transformation
log_norm_counts <- log2(normalized_counts + 1)

cat("Log transform count matrix dimensions:", dim(log_norm_counts), "\n")

## Log transform count matrix dimensions: 60660 427

cat("Log transformation range:", range(log_norm_counts), "\n\n")

## Log transformation range: 0 23.74122

# Apply VST (recommended for >30 samples) - Applies variance-stabilizing tran
sformation, 'blind' Ignores experimental design
vsd <- vst(dds, blind = TRUE)
vst_counts <- assay(vsd)

cat("VST count matrix dimensions:", dim(vst_counts), "\n")

## VST count matrix dimensions: 60660 427

cat("VST value range:", range(vst_counts), "\n\n")

## VST value range: 3.184037 23.74122
```

2.3: Z-score transformation

```
# Z-score the log-transformed counts (by gene)
z_log <- t(scale(t(log_norm_counts)))

cat("Z-scored log2 matrix dimensions:", dim(z_log), "\n")

## Z-scored log2 matrix dimensions: 60660 427

cat("Z-scored log2 range:", range(z_log, na.rm = TRUE), "\n\n")

## Z-scored log2 range: -13.30718 20.61558
```

```r
# Z-score the VST-transformed counts (by gene)
z_vst <- t(scale(t(vst_counts)))

cat("Z-scored VST matrix dimensions:", dim(z_vst), "\n")
```

```
## Z-scored VST matrix dimensions: 60660 427
```

```r
cat("Z-scored VST range:", range(z_vst, na.rm = TRUE), "\n\n")
```

```
## Z-scored VST range: -12.22298 20.61558
```

==========

INVESTIGATION 1: Global Expression-Based Clustering

RQ 2.1: Do patients cluster into distinct phenotype subtypes?

==========

Filter the most variable genes

Running distance on 60660 observations was too computationally intensive. We decided to only keep the top 3000 genes since PCA will explain only part of the variance and variance will be mostly explained by those highly variable 3000 genes. It was chosen arbitrarily

```r
# compute variance on raw data
gene_vars <- apply(normalized_counts, 1, var)


#Rank the genes by variance
top_genes <- names(sort(gene_vars, decreasing = TRUE))[1:3000]

# Create a subset matrix with the top 3000 genes
expr_z_top <- z_vst[top_genes, ]
```

Plot PCA and look for distinct group

```r
pca_res <- prcomp(t(expr_z_top), scale = TRUE)

# Percentage variance explained
```

```r
pve <- (pca_res$sdev^2) / sum(pca_res$sdev^2)

# Create and save Scree plot
scree_plot <- ggplot(data.frame(PC = 1:10, Var = pve[1:10]),
       aes(x = PC, y = Var)) +
  geom_bar(stat = "identity") +
  ggtitle("Scree Plot: Variance Explained by PCA Components") +
  ylab("Proportion of Variance Explained")

ggsave("scree_plot.png", scree_plot, width = 7, height = 5, dpi = 300)

# Create and save PCA scatter plot
pca_df <- data.frame(
  Sample = rownames(t(expr_z_top)),
  PC1 = pca_res$x[,1],
  PC2 = pca_res$x[,2]
)

pca_scatter <- ggplot(pca_df, aes(x = PC1, y = PC2)) +
  geom_point(alpha = 0.7) +
  ggtitle("PCA of KIRC Patients (Top 3000 Variance Genes)") +
  xlab(paste0("PC1 (", round(pve[1] * 100, 1), "%)")) +
  ylab(paste0("PC2 (", round(pve[2] * 100, 1), "%)")) +
  theme_minimal()

ggsave("pca_scatter.png", pca_scatter, width = 7, height = 6, dpi = 300)
```

Select PCs explaining 80% variance

```r
# Compute cumulative variance explained
cumvar <- cumsum(pve)

# Find how many PCs are needed to reach ≥80% variance
Top_pc <- which(cumvar >= 0.80)[1]
cat("Number of PCs explaining ≥80% variance:", Top_pc, "\n")
```

```
## Number of PCs explaining ≥80% variance: 32
```

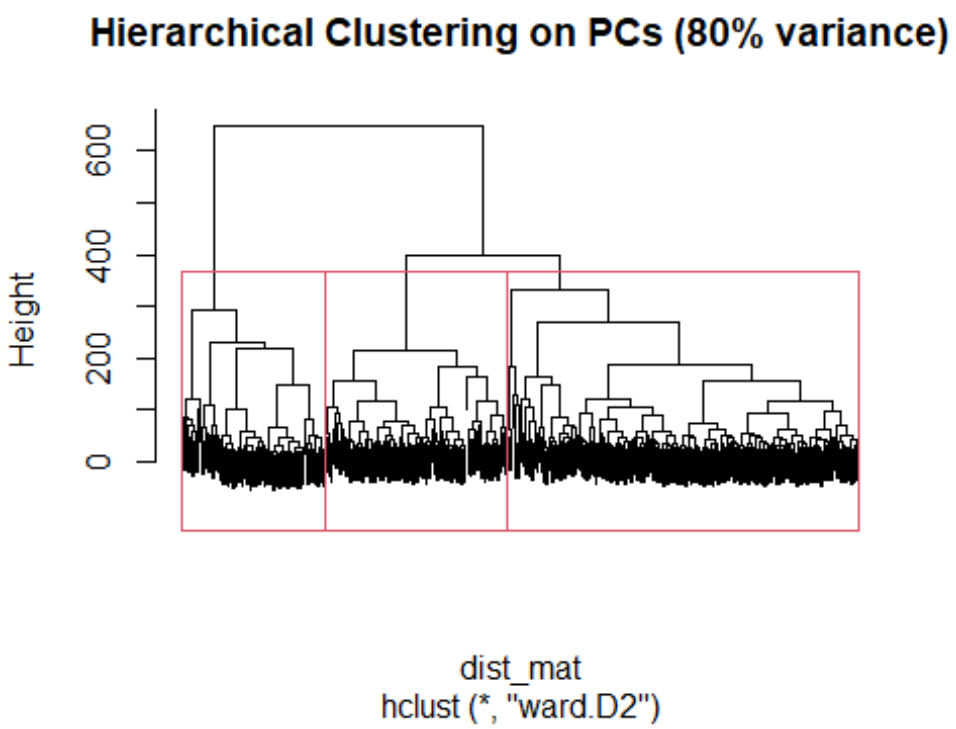
Extract the PC score

```r
# PCA scores (samples × PCs)
pc_scores <- pca_res$x[, 1:Top_pc]
```

Cluster on the main PCs

```r
# Compute distance on reduced PC space
dist_mat <- dist(pc_scores)
```

```r
# Ward's hierarchical clustering
hc <- hclust(dist_mat, method = "ward.D2")

# Plot dendrogram
plot(hc, labels = FALSE, main = "Hierarchical Clustering on PCs (80% variance
)")
rect.hclust(hc, k = 3, border = 2)
```




```r
png("dendrogram.png", width = 2400, height = 1800, res = 300)
plot(hc, labels = FALSE, main = "Hierarchical Clustering on PCs (80% variance
)")
rect.hclust(hc, k = 3, border = 2)
dev.off()

## png
##   2
```

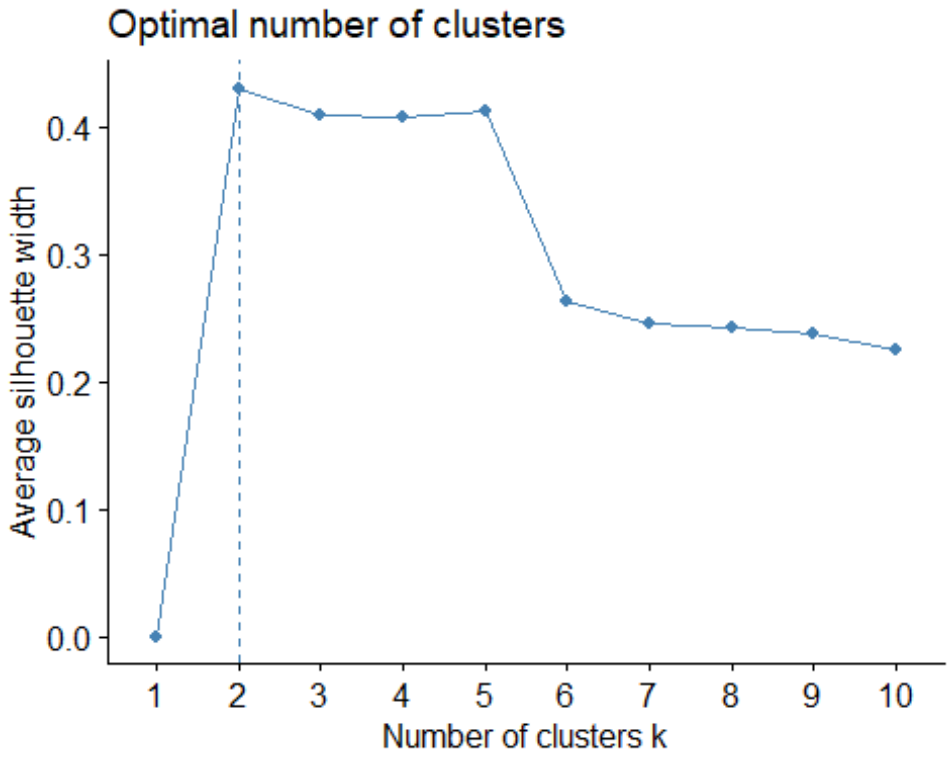
#Cutting Dendogram We decided to cut the dendrogram at 6 since there is 6 subtypes of KIRC

```r
clusters <- cutree(hc, k = 3)
table(clusters)

## clusters
##   1   2   3
## 222 115  90
```

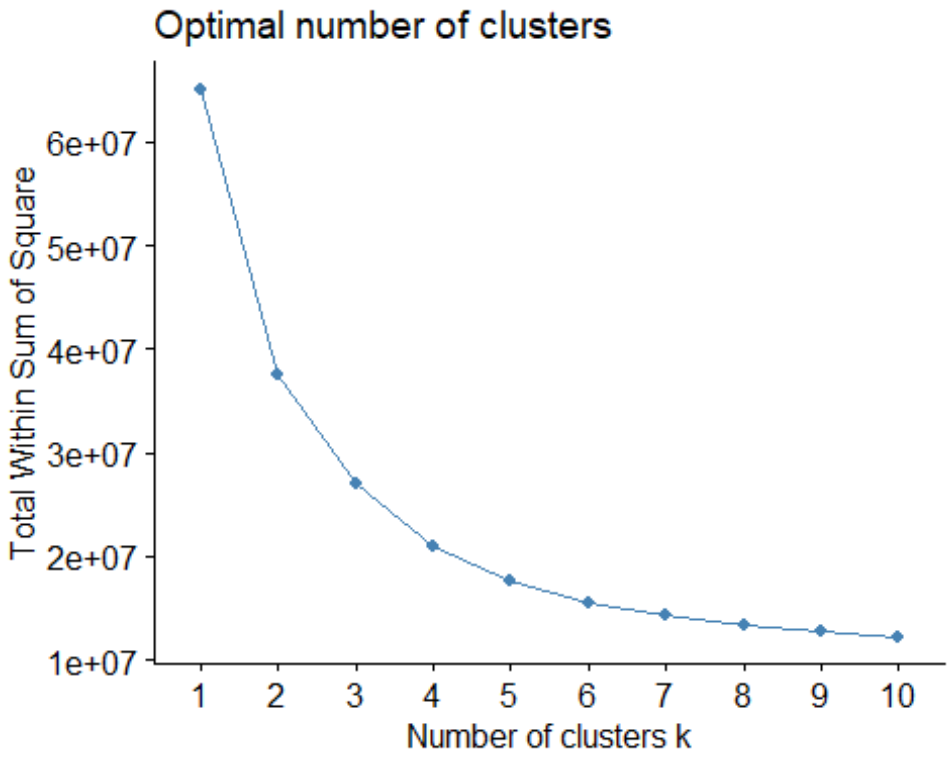
Plot Overlapping Elbow and silhouette graphs

Do not pick 2, Since it was proven online that they might be artifacts of batch effects

```
# install.packages("factoextra")
library(factoextra)

## Warning: package 'factoextra' was built under R version 4.5.2

## Welcome! Want to learn more? See two factoextra-related books at https://g
oo.gl/ve3WBa

# Silhouette width
silhouette <- fviz_nbclust(as.matrix(dist_mat), hcut, method = "silhouette",
hc_method = "ward.D2")
silhouette
```




```
# Elbow plot
withinsumsquares <- fviz_nbclust(as.matrix(dist_mat), hcut, method = "wss", h
c_method = "ward.D2")
withinsumsquares
```




```
ggsave("silhouette_plot.png", silhouette, width = 7, height = 5, dpi = 300)
ggsave("elbow_plot.png", withinsumsquares, width = 7, height = 5, dpi = 300)
```

==========

INVESTIGATION 2: Survival across the patient clusters is different.

RQ 2.2: Do expression-based clusters differ in patient outcomes?

==========

Create a dataframe that contains the DSS and the cluster labels. Rowname is patient ID

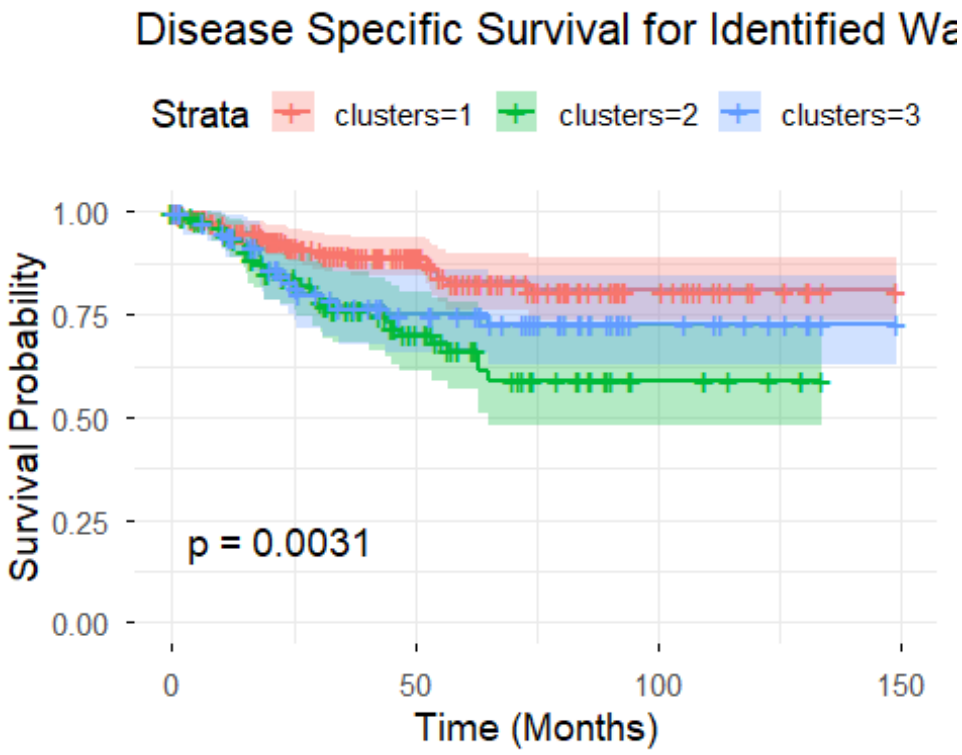
```
# Create a dataframe that contains the cluster and the patient ID
clustering_data <- data.frame(
```

```
  clusters = clusters,
  PATIENT_ID = names(clusters)
)

# transform clustering data into standard patient ID
clustering_data$PATIENT_ID <- gsub("\\.", "-", substr(clustering_data$PATIENT
_ID, 1, 12))

rownames(clustering_data) <- NULL

clinical_data_subset <- data_clinical_patient[
 data_clinical_patient$PATIENT_ID %in% clustering_data$PATIENT_ID,
 c("PATIENT_ID", "DSS_STATUS", "DSS_MONTHS")
]

# transform the DSS status to binary
clinical_data_subset$DSS_STATUS <- substr(clinical_data_subset$DSS_STATUS, 1,
1)

merged_data <- merge(
 clustering_data,
 clinical_data_subset,
 by = "PATIENT_ID",
 all.x = FALSE,
 all.y = FALSE
)
```

Survival Analysis Across different clusters

```
surv_obj_ward_DSS <- Surv(time = as.numeric(merged_data$DSS_MONTHS), event= a
s.numeric(merged_data$DSS_STATUS))

# KM plot for ward.D2 clustering
ward_d2_DSS <- surv_fit(surv_obj_ward_DSS ~ clusters, data = merged_data)
ward_d2_DSS_plot <- ggsurvplot(ward_d2_DSS,
           data = merged_data,
           pval = TRUE,
           conf.int = TRUE,
           title = "Disease Specific Survival for Identified Ward.D2 Hierarch
ical Clusters",
           xlab = "Time (Months)",
           ylab = "Survival Probability",
           ggtheme = theme_minimal(base_size = 14))

## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## ℹ Please use `linewidth` instead.
## ℹ The deprecated feature was likely used in the ggpubr package.
##   Please report the issue at <https://github.com/kassambara/ggpubr/issues>
.
```

```
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.

ward_d2_DSS_plot
```




```
ggsave("DSSacrossclusters.png", ward_d2_DSS_plot$plot, width=7, height=3, dpi
=300)
```

Since we see 3 clear outcomes in survival. This is good since there is 3 histological subtypes

Assign the clusters to respective patients and their data

```
# Create a dataframe that contains the cluster and the patient ID
clustering_data <- data.frame(
  clusters = clusters,
  PATIENT_ID = names(clusters)
)

# Transform clustering data into standard patient ID
clustering_data$PATIENT_ID <- gsub("\\.", "-", substr(clustering_data$PATIENT
_ID, 1, 12))
rownames(clustering_data) <- NULL

# Extract relevant clinical data
```

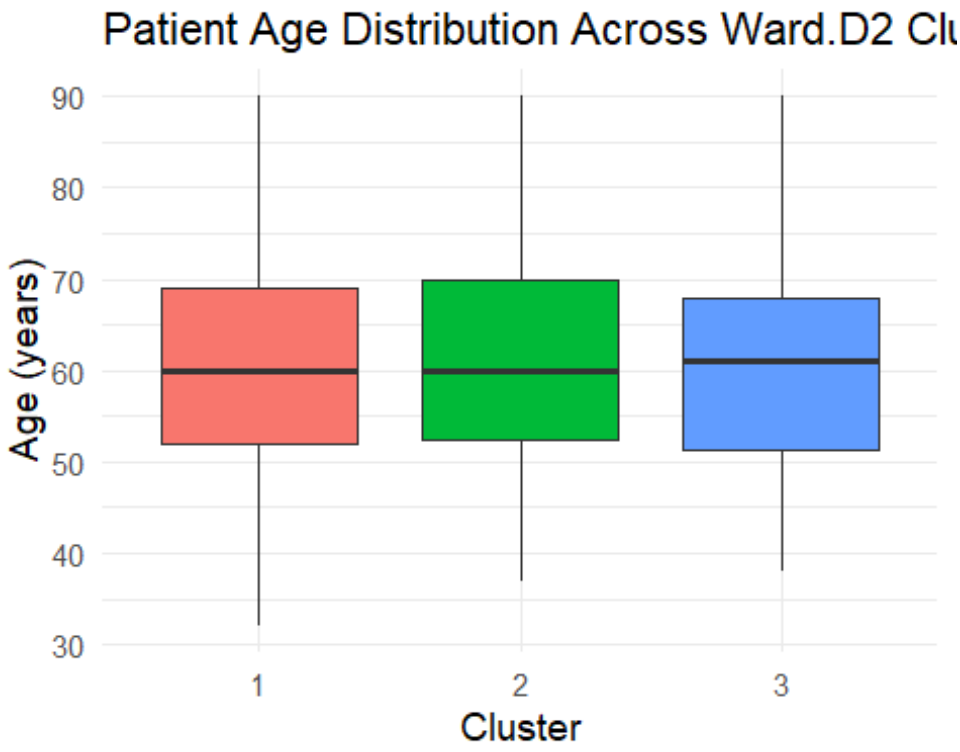
```
clinical_data_subset <- data_clinical_patient[
  data_clinical_patient$PATIENT_ID %in% clustering_data$PATIENT_ID,
  c("PATIENT_ID", "DSS_STATUS", "DSS_MONTHS", "AGE", "AJCC_PATHOLOGIC_TUMOR_S
TAGE")
]

# Transform the DSS status to binary (0 = alive, 1 = dead)
clinical_data_subset$DSS_STATUS <- substr(clinical_data_subset$DSS_STATUS, 1,
1)

# Transform the tumor stage
clinical_data_subset$AJCC_PATHOLOGIC_TUMOR_STAGE <- sapply(clinical_data_subs
et$AJCC_PATHOLOGIC_TUMOR_STAGE, function(x) {
  if (grepl("stage", x, ignore.case = TRUE)) {
            as.numeric(as.roman(gsub("stage ", "", x, ignore.case = TRUE)))
  } else {
    NA
  }
})

# Merge clustering and clinical data
merged_data <- merge(
  clustering_data,
  clinical_data_subset,
  by = "PATIENT_ID",
  all.x = FALSE,
  all.y = FALSE
)
```

Look for diffences in age - ANOVA. Code adapted from part 1 of the project.

```
# One-way ANOVA for age
merged_data$clusters <- as.factor(merged_data$clusters)

anova_result_Ward_D2 <- aov(merged_data$AGE ~ clusters, data = merged_data)
summary(anova_result_Ward_D2)

##              Df Sum Sq Mean Sq F value Pr(>F)
## clusters      2     32   16.24   0.114  0.893
## Residuals   424  60571  142.86

TukeyHSD(anova_result_Ward_D2)

##   Tukey multiple comparisons of means
##     95% family-wise confidence level
##
## Fit: aov(formula = merged_data$AGE ~ clusters, data = merged_data)
```

```
##
## $clusters
##           diff       lwr      upr     p adj
## 2-1  0.6336467 -2.596115 3.863409 0.8893148
## 3-1  0.3867868 -3.126071 3.899644 0.9637093
## 3-2 -0.2468599 -4.203146 3.709427 0.9881970
```

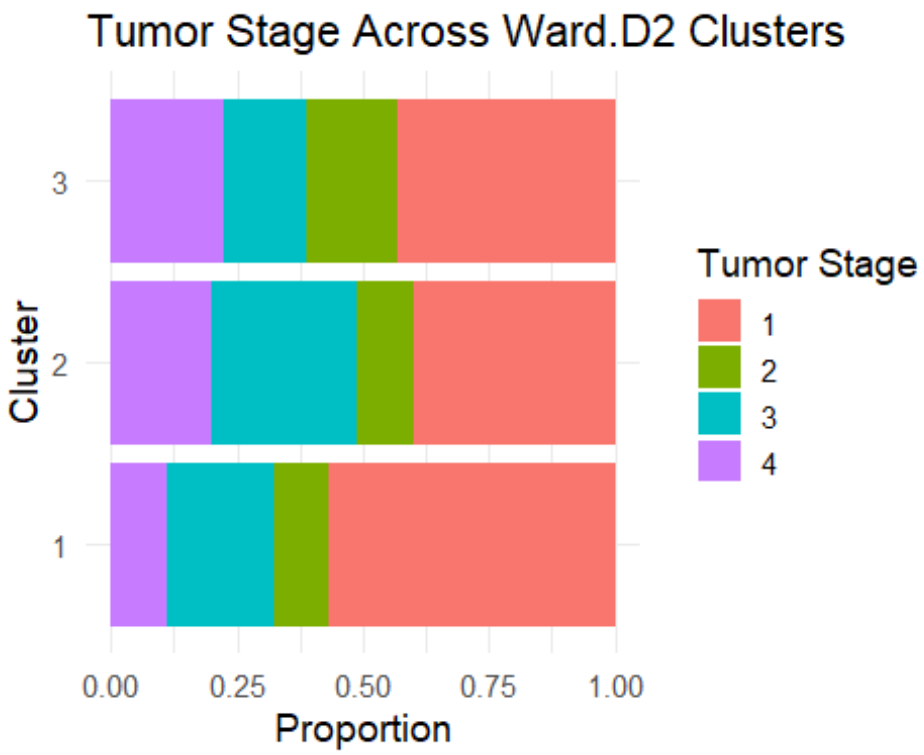
```
anova_ward_d2_plot <- ggplot(merged_data, aes(x = clusters, y = AGE, fill = c
lusters)) +
  geom_boxplot() +
  labs(
    title = "Patient Age Distribution Across Ward.D2 Clusters",
    x = "Cluster",
    y = "Age (years)"
  ) +
  theme_minimal(base_size = 14) +
  theme(legend.position = "none")

# Plot
anova_ward_d2_plot
```




```
ggsave("AgeDistributionAcrossClusters_WardD2.png", anova_ward_d2_plot, width=
7, height=3, dpi=300)
```

What about Tumor stage?

```
merged_data$AJCC_PATHOLOGIC_TUMOR_STAGE <- as.factor(merged_data$AJCC_PATHOLO
GIC_TUMOR_STAGE)

# Contingency table
table_stage_cluster_wardd2 <- table(merged_data$clusters, merged_data$AJCC_PA
THOLOGIC_TUMOR_STAGE)
table_stage_cluster_wardd2

##
##       1   2   3   4
##   1 126  24  47  25
##   2  46  13  33  23
##   3  39  16  15  20

# Chi-square Test
chisq_test_wardd2 <- chisq.test(table_stage_cluster_wardd2)
chisq_test_wardd2

##
##  Pearson's Chi-squared test
##
## data:  table_stage_cluster_wardd2
## X-squared = 17.815, df = 6, p-value = 0.006711

# Visualization
chi_test_plot_ward2 <- ggplot(merged_data, aes(x = clusters, fill = AJCC_PATH
OLOGIC_TUMOR_STAGE)) +
  geom_bar(position = "fill") +  # fill = proportion per cluster
  coord_flip() +
  ylab("Proportion") +
  xlab("Cluster") +
  labs(title = "Tumor Stage Across Ward.D2 Clusters", fill = "Tumor Stage") +
  theme_minimal(base_size = 14)

#plot
chi_test_plot_ward2
```




```
#save
ggsave("TumorStageAcrossClusters_WardD2.png",chi_test_plot_ward2 , width=7, h
eight=3, dpi=300)
```

==========

INVESTIGATION 3: Check DE across clusters

RQ 2.2: What gene vary across clusters? Are they Significative of KIRC subtype?

==========

This step was made by claudeAI usign the prompt(s):“Read this + Previous code, and make a DE heatmap using pheatmap”

```
cat("Merged data dimensions:", dim(merged_data), "\n")
```

```
## Merged data dimensions: 427 6
```

```
cat("Cluster distribution:\n")
```

```
## Cluster distribution:

print(table(merged_data$clusters))

##
##   1   2   3
## 222 115  90

cat("\n")

#==========================================================================
# Run DESeq2 for Differential Expression Between Clusters
# ==========================================================================

# Prepare DESeq2 object with cluster information
# Use the original sample names (with dots/full barcodes)
coldata_clusters <- data.frame(
  sample = colnames(count_matrix),
  cluster = clusters[colnames(count_matrix)]
)
rownames(coldata_clusters) <- coldata_clusters$sample
coldata_clusters$cluster <- as.factor(coldata_clusters$cluster)

# Create DESeq2 dataset with cluster design
dds_de <- DESeqDataSetFromMatrix(
  countData = count_matrix,
  colData = coldata_clusters,
  design = ~ cluster
)

# Filter low-count genes (at least 10 reads in at least 10% of samples)
keep <- rowSums(counts(dds_de) >= 10) >= ncol(dds_de) * 0.1
dds_de <- dds_de[keep, ]
cat("Genes retained after filtering:", nrow(dds_de), "\n\n")

## Genes retained after filtering: 26443

# Run DESeq2
dds_de <- DESeq(dds_de)

 ==========================================================================
# Extract Top Differentially Expressed Genes
#==========================================================================

# Get all pairwise comparisons
comparisons <- list(
  c1_vs_c2 = results(dds_de, contrast = c("cluster", "1", "2")),
  c1_vs_c3 = results(dds_de, contrast = c("cluster", "1", "3")),
  c2_vs_c3 = results(dds_de, contrast = c("cluster", "2", "3"))
)
```

```r
# Find significant DEGs for each comparison (padj < 0.05, |log2FC| > 1)
sig_genes_list <- lapply(comparisons, function(res) {
  sig <- res[which(res$padj < 0.05 & abs(res$log2FoldChange) > 1), ]
  rownames(sig)
})

# Combine all significant genes
all_sig_genes <- unique(unlist(sig_genes_list))
cat("Total unique DEGs across all comparisons:", length(all_sig_genes), "\n")
```

```
## Total unique DEGs across all comparisons: 11683
```

```r
# Select top genes by variance if there are too many
if (length(all_sig_genes) > 500) {
  gene_vars_sig <- apply(vst_counts[all_sig_genes, ], 1, var)
  top_sig_genes <- names(sort(gene_vars_sig, decreasing = TRUE))[1:500]
  cat("Selected top 500 DEGs by variance\n\n")
} else {
  top_sig_genes <- all_sig_genes
}
```

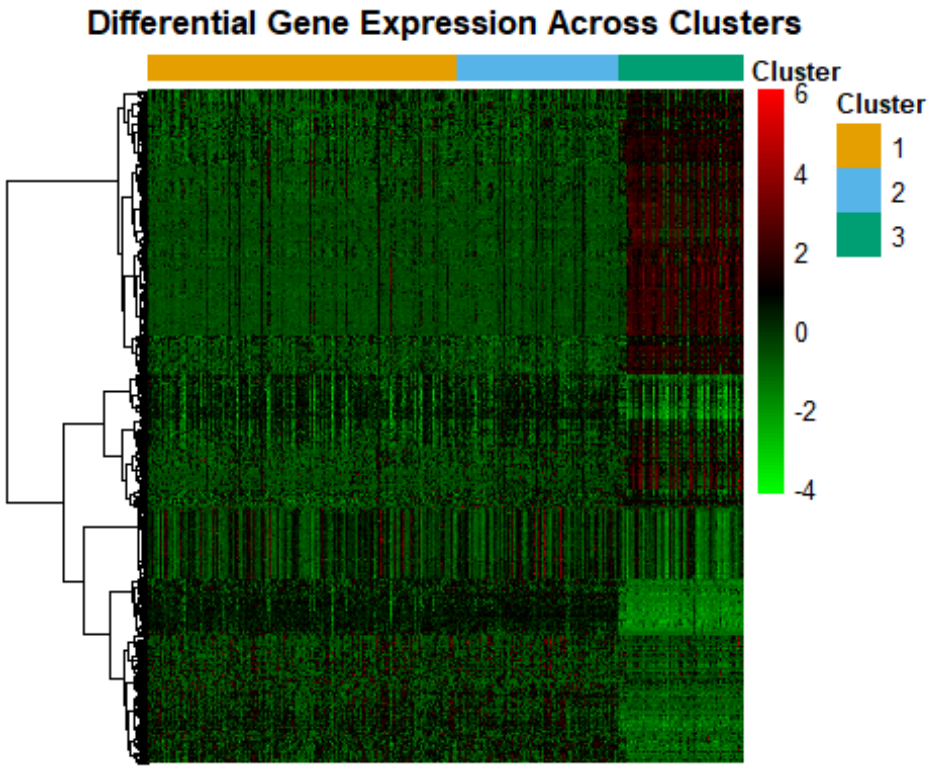
```
## Selected top 500 DEGs by variance
```

Create Heatmap

```r
# Extract VST counts for top DEGs
heatmap_data <- vst_counts[top_sig_genes, ]

# Z-score normalize by gene (rows)
heatmap_data_scaled <- t(scale(t(heatmap_data)))

# Order samples by cluster
sample_order <- order(clusters)
heatmap_data_scaled <- heatmap_data_scaled[, sample_order]
clusters_ordered <- clusters[sample_order]

# Create annotation for samples (columns)
annotation_col <- data.frame(
  Cluster = as.factor(clusters_ordered)
)
rownames(annotation_col) <- colnames(heatmap_data_scaled)

# Define colors for clusters
cluster_colors <- list(
  Cluster = c("1" = "#E69F00", "2" = "#56B4E9", "3" = "#009E73")
)
```

Save the image

```
# Create the heatmap
DEheatmap <- pheatmap(
  heatmap_data_scaled,
  scale = "none",  # Already scaled
  cluster_rows = TRUE,  # Cluster genes
  cluster_cols = FALSE,  # Keep sample order by cluster
  clustering_distance_rows = "euclidean",
  clustering_method = "ward.D2",
  annotation_col = annotation_col,
  annotation_colors = cluster_colors,
  show_colnames = FALSE,  # Too many samples
  show_rownames = FALSE,  # Too many genes
  color = colorRampPalette(c("green", "black", "red"))(100),
  main = "Differential Gene Expression Across Clusters",
  fontsize = 10,
  border_color = NA,
  #filename = "DEpercluster.png"  # Add this parameter
)
```



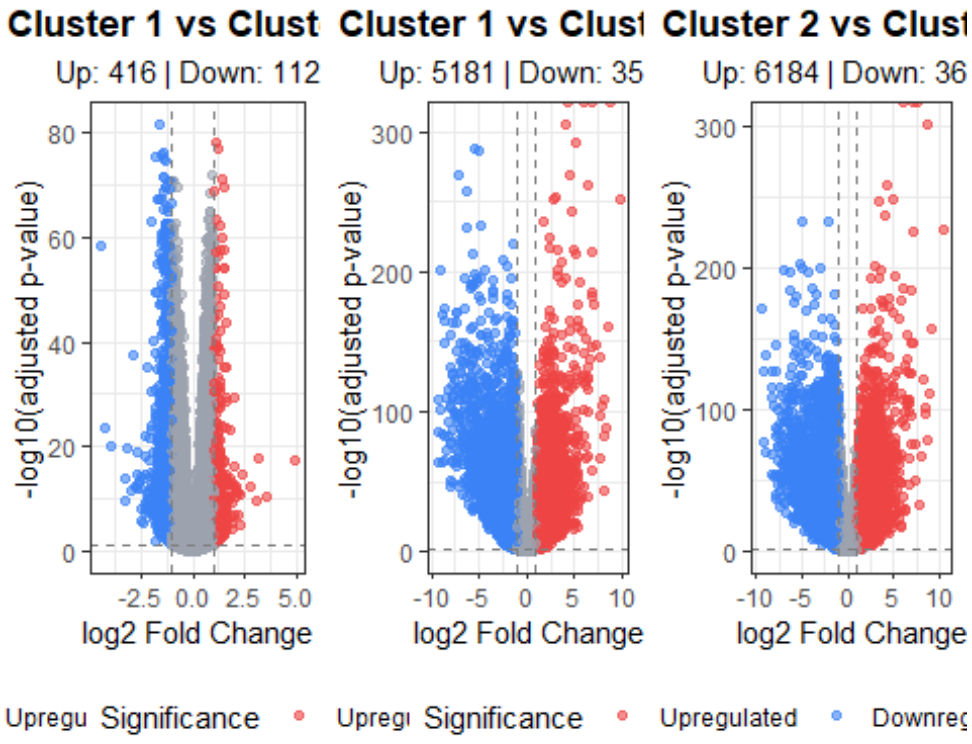
Which genes are the most differentially expressed? Code belwo was generated with ClaudeAI and the prompts: I want as simple as possible, in R so I can see hte comparisons. + Code aboe

```
library(ggplot2)
library(gridExtra)
```

```
##
## Attaching package: 'gridExtra'

## The following object is masked from 'package:dplyr':
##
##     combine

## The following object is masked from 'package:Biobase':
##
##     combine

## The following object is masked from 'package:BiocGenerics':
##
##     combine
```

```
# Function to create volcano plot
make_volcano <- function(res, title) {
  # Prepare data
  df <- as.data.frame(res)
  df$gene <- rownames(df)
  df$neglog10padj <- -log10(df$padj)

  # Add significance category
  df$significance <- "Not Significant"
  df$significance[df$padj < 0.05 & df$log2FoldChange > 1] <- "Upregulated"
  df$significance[df$padj < 0.05 & df$log2FoldChange < -1] <- "Downregulated"
  df$significance <- factor(df$significance,
                            levels = c("Upregulated", "Downregulated", "Not Si
gnificant"))

  # Remove NA values
  df <- df[!is.na(df$padj) & !is.na(df$log2FoldChange), ]

  # Count genes in each category
  n_up <- sum(df$significance == "Upregulated")
  n_down <- sum(df$significance == "Downregulated")

  # Create plot
  p <- ggplot(df, aes(x = log2FoldChange, y = neglog10padj, color = significa
nce)) +
    geom_point(alpha = 0.6, size = 1.5) +
    scale_color_manual(values = c("Upregulated" = "#ef4444",
                                  "Downregulated" = "#3b82f6",
                                  "Not Significant" = "#9ca3af")) +
    geom_vline(xintercept = c(-1, 1), linetype = "dashed", color = "gray50")
+
    geom_hline(yintercept = -log10(0.05), linetype = "dashed", color = "gray5
0") +
    labs(title = title,
         subtitle = paste0("Up: ", n_up, " | Down: ", n_down),
```

```
         x = "log2 Fold Change",
         y = "-log10(adjusted p-value)",
         color = "Significance") +
    theme_bw() +
    theme(plot.title = element_text(hjust = 0.5, face = "bold"),
          plot.subtitle = element_text(hjust = 0.5),
          legend.position = "bottom")

  return(p)
}

# Create volcano plots for each comparison
p1 <- make_volcano(comparisons$c1_vs_c2, "Cluster 1 vs Cluster 2")
p2 <- make_volcano(comparisons$c1_vs_c3, "Cluster 1 vs Cluster 3")
p3 <- make_volcano(comparisons$c2_vs_c3, "Cluster 2 vs Cluster 3")

# Display all three plots
grid.arrange(p1, p2, p3, ncol = 3)
```




```
# Optional: Save as high-resolution figure
ggsave("volcano_c1_vs_c2.png", p1, width = 8, height = 6, dpi = 300)
ggsave("volcano_c1_vs_c3.png", p2, width = 8, height = 6, dpi = 300)
ggsave("volcano_c2_vs_c3.png", p3, width = 8, height = 6, dpi = 300)
```

Which genes (by name) are the most differentially expressed?

```
cat("\n=== Differential Expression Summary ===\n")

##
## === Differential Expression Summary ===

for(comp_name in names(comparisons)) {
  res <- comparisons[[comp_name]]
  df <- as.data.frame(res)
  df$gene <- rownames(df)
  df <- df[!is.na(df$padj), ]

  n_up <- sum(df$padj < 0.05 & df$log2FoldChange > 1, na.rm = TRUE)
  n_down <- sum(df$padj < 0.05 & df$log2FoldChange < -1, na.rm = TRUE)
  n_total_sig <- sum(df$padj < 0.05, na.rm = TRUE)

  cat(sprintf("\n%s:\n", comp_name))
  cat(sprintf("  Upregulated (padj<0.05, log2FC>1): %d\n", n_up))
  cat(sprintf("  Downregulated (padj<0.05, log2FC<-1): %d\n", n_down))
  cat(sprintf("  Total significant (padj<0.05): %d\n", n_total_sig))

  # Get top upregulated genes
  cat("\n  Top 10 UPREGULATED genes:\n")
  top_up <- df[df$padj < 0.05 & df$log2FoldChange > 0, ]
  top_up <- top_up[order(top_up$log2FoldChange, decreasing = TRUE), ]
  if(nrow(top_up) > 0) {
    for(i in 1:min(10, nrow(top_up))) {
      cat(sprintf("    %2d. %-15s | log2FC: %6.2f | padj: %.2e\n",
                  i, top_up$gene[i], top_up$log2FoldChange[i], top_up$padj[i]
))
    }
  } else {
    cat("    No significantly upregulated genes\n")
  }

  # Get top downregulated genes
  cat("\n  Top 10 DOWNREGULATED genes:\n")
  top_down <- df[df$padj < 0.05 & df$log2FoldChange < 0, ]
  top_down <- top_down[order(top_down$log2FoldChange, decreasing = FALSE), ]
  if(nrow(top_down) > 0) {
    for(i in 1:min(10, nrow(top_down))) {
      cat(sprintf("    %2d. %-15s | log2FC: %6.2f | padj: %.2e\n",
                  i, top_down$gene[i], top_down$log2FoldChange[i], top_down$p
adj[i]))
    }
  } else {
    cat("    No significantly downregulated genes\n")
```

```
  }
  cat("\n", strrep("-", 70), "\n")
}
```

```
##
## c1_vs_c2:
##   Upregulated (padj<0.05, log2FC>1): 416
##   Downregulated (padj<0.05, log2FC<-1): 1120
##   Total significant (padj<0.05): 17275
##
##   Top 10 UPREGULATED genes:
##      1. ENSG00000105141.6 | log2FC:   4.93 | padj: 4.23e-18
##      2. ENSG00000263146.2 | log2FC:   3.55 | padj: 2.91e-11
##      3. ENSG00000070915.10 | log2FC:   3.11 | padj: 1.27e-18
##      4. ENSG00000256463.8 | log2FC:   3.04 | padj: 1.90e-10
##      5. ENSG00000271945.2 | log2FC:   2.80 | padj: 3.06e-13
##      6. ENSG00000134258.17 | log2FC:   2.32 | padj: 1.81e-15
##      7. ENSG00000168143.9 | log2FC:   2.29 | padj: 1.26e-11
##      8. ENSG00000181092.10 | log2FC:   2.27 | padj: 1.16e-05
##      9. ENSG00000130294.17 | log2FC:   2.22 | padj: 5.78e-12
##     10. ENSG00000287702.1 | log2FC:   2.14 | padj: 1.49e-07
##
##   Top 10 DOWNREGULATED genes:
##      1. ENSG00000225972.1 | log2FC:  -4.43 | padj: 5.35e-59
##      2. ENSG00000150201.15 | log2FC:  -4.19 | padj: 3.35e-24
##      3. ENSG00000223783.1 | log2FC:  -3.89 | padj: 1.12e-20
##      4. ENSG00000261667.1 | log2FC:  -3.29 | padj: 1.80e-10
##      5. ENSG00000253802.2 | log2FC:  -3.27 | padj: 1.52e-14
##      6. ENSG00000170367.5 | log2FC:  -3.13 | padj: 1.13e-20
##      7. ENSG00000254475.1 | log2FC:  -3.00 | padj: 3.09e-12
##      8. ENSG00000262902.1 | log2FC:  -2.90 | padj: 2.02e-38
##      9. ENSG00000261514.1 | log2FC:  -2.74 | padj: 5.23e-13
##     10. ENSG00000287407.1 | log2FC:  -2.70 | padj: 9.38e-13
##
##  ----------------------------------------------------------------------
##
## c1_vs_c3:
##   Upregulated (padj<0.05, log2FC>1): 5181
##   Downregulated (padj<0.05, log2FC<-1): 3520
##   Total significant (padj<0.05): 21326
##
##   Top 10 UPREGULATED genes:
##      1. ENSG00000164434.12 | log2FC:   9.72 | padj: 4.41e-252
##      2. ENSG00000224490.5 | log2FC:   8.66 | padj: 0.00e+00
##      3. ENSG00000225174.2 | log2FC:   8.60 | padj: 8.93e-161
##      4. ENSG00000286085.1 | log2FC:   8.40 | padj: 1.33e-89
##      5. ENSG00000168619.16 | log2FC:   8.09 | padj: 6.90e-110
##      6. ENSG00000168333.14 | log2FC:   8.01 | padj: 5.88e-84
##      7. ENSG00000214978.7 | log2FC:   7.99 | padj: 1.92e-44
##      8. ENSG00000251320.1 | log2FC:   7.78 | padj: 4.23e-105
```

```
##      9. ENSG00000178776.5 | log2FC:   7.75 | padj: 1.96e-139
##     10. ENSG00000248362.1 | log2FC:   7.71 | padj: 1.40e-97
##
##   Top 10 DOWNREGULATED genes:
##      1. ENSG00000174562.13 | log2FC:  -9.36 | padj: 8.83e-109
##      2. ENSG00000279460.1 | log2FC:  -9.19 | padj: 1.03e-85
##      3. ENSG00000169344.15 | log2FC:  -9.19 | padj: 4.35e-64
##      4. ENSG00000086159.13 | log2FC:  -9.12 | padj: 6.17e-202
##      5. ENSG00000225064.1 | log2FC:  -9.00 | padj: 9.46e-102
##      6. ENSG00000167580.8 | log2FC:  -8.89 | padj: 4.17e-64
##      7. ENSG00000168269.10 | log2FC:  -8.81 | padj: 2.65e-85
##      8. ENSG00000188175.10 | log2FC:  -8.81 | padj: 4.57e-170
##      9. ENSG00000249601.2 | log2FC:  -8.68 | padj: 7.01e-150
##     10. ENSG00000140519.14 | log2FC:  -8.66 | padj: 1.94e-165
##
##  ----------------------------------------------------------------------
##
## c2_vs_c3:
##   Upregulated (padj<0.05, log2FC>1): 6184
##   Downregulated (padj<0.05, log2FC<-1): 3649
##   Total significant (padj<0.05): 21766
##
##   Top 10 UPREGULATED genes:
##      1. ENSG00000164434.12 | log2FC:  10.38 | padj: 1.32e-227
##      2. ENSG00000178776.5 | log2FC:   9.21 | padj: 1.13e-157
##      3. ENSG00000248362.1 | log2FC:   8.97 | padj: 5.95e-112
##      4. ENSG00000224490.5 | log2FC:   8.73 | padj: 7.90e-302
##      5. ENSG00000168333.14 | log2FC:   8.71 | padj: 1.02e-79
##      6. ENSG00000188373.5 | log2FC:   8.49 | padj: 9.54e-98
##      7. ENSG00000251320.1 | log2FC:   8.36 | padj: 3.34e-102
##      8. ENSG00000225174.2 | log2FC:   8.26 | padj: 4.38e-122
##      9. ENSG00000286085.1 | log2FC:   8.00 | padj: 1.65e-67
##     10. ENSG00000214978.7 | log2FC:   7.70 | padj: 6.26e-34
##
##   Top 10 DOWNREGULATED genes:
##      1. ENSG00000086159.13 | log2FC:  -9.51 | padj: 3.93e-172
##      2. ENSG00000225064.1 | log2FC:  -9.15 | padj: 6.42e-78
##      3. ENSG00000249601.2 | log2FC:  -9.11 | padj: 2.85e-128
##      4. ENSG00000168269.10 | log2FC:  -9.06 | padj: 5.24e-71
##      5. ENSG00000188175.10 | log2FC:  -9.05 | padj: 9.72e-140
##      6. ENSG00000174562.13 | log2FC:  -8.39 | padj: 5.90e-69
##      7. ENSG00000173612.10 | log2FC:  -8.36 | padj: 9.60e-55
##      8. ENSG00000167748.11 | log2FC:  -8.29 | padj: 9.25e-128
##      9. ENSG00000105929.16 | log2FC:  -8.21 | padj: 1.46e-116
##     10. ENSG00000265460.6 | log2FC:  -8.20 | padj: 3.49e-109
##
##  ----------------------------------------------------------------------
```

==========

INVESTIGATION 4:

==========

```
## Get Initial Count Matrix
# Create count matrix (genes as rows, samples as columns)
count_matrix <- as.matrix(RNAseq_KIRC[, -1])  # Remove gene name column
# Assuming your Ensembl IDs are in the first column of RNAseq_KIRC
ensemble_ids <- RNAseq_KIRC[, 1]
# Fix common Ensembl ID format issue (remove version number like ".1")
# This ensures a better match rate with the annotation package
ensemble_ids_clean <- sub("\\.\\d+$", "", ensemble_ids)
rownames(count_matrix) <- ensemble_ids_clean

cat("Initial count matrix dimensions (by Ensembl ID):", dim(count_matrix), "\
n")

## Initial count matrix dimensions (by Ensembl ID): 60660 427

# Map Ensembl IDs to Gene Symbols
gene_map <- select(
  org.Hs.eg.db,
  keys = rownames(count_matrix),
  columns = c("SYMBOL"),
  keytype = "ENSEMBL"
)

## 'select()' returned 1:many mapping between keys and columns

# Filter out Ensembl IDs that didn't map to a Gene Symbol
gene_map <- gene_map %>% filter(!is.na(SYMBOL))

# Handle one-to-many mappings (Multiple Ensembl IDs map to one Symbol)
# For simplicity, we keep the first occurrence of a symbol
gene_map <- gene_map %>%
  distinct(ENSEMBL, .keep_all = TRUE) %>% # Keep one SYMBOL per ENSEMBL
  distinct(SYMBOL, .keep_all = TRUE) # Keep one row per unique SYMBOL

cat("Number of unique Ensembl IDs mapped to a unique Symbol:", nrow(gene_map)
, "\n")

## Number of unique Ensembl IDs mapped to a unique Symbol: 36585

# Filter and Rename the Count Matrix

# Filter the count matrix to include only mapped genes
```

```r
mapped_count_matrix <- count_matrix[gene_map$ENSEMBL, ]

# Assign the Gene Symbols as row names
rownames(mapped_count_matrix) <- gene_map$SYMBOL

cat("Final count matrix dimensions (by Gene Symbol):", dim(mapped_count_matri
x), "\n")
```

```
## Final count matrix dimensions (by Gene Symbol): 36585 427
```

```r
# Extract sample IDs from your count matrix column names
sample_ids <- colnames(mapped_count_matrix)

# Clean up sample IDs to match patient IDs (remove any extra characters)
patient_ids <- gsub("\\.", "-", substr(sample_ids, 1, 12))

# Create age_info by matching with clinical data
age_info <- data.frame(
  sample_id = sample_ids,
  patient_id = patient_ids,
  stringsAsFactors = FALSE
)

# Merge with clinical data to get age information
age_info <- merge(
  age_info,
  data_clinical_patient[, c("PATIENT_ID", "AGE")],
  by.x = "patient_id",
  by.y = "PATIENT_ID",
  all.x = TRUE
)

# Create age groups based on the AGE column
# Categorize into "0-79" and "80+" groups
age_info$age <- ifelse(age_info$AGE < 80, "0-79", "80+")

# Recode age (make under80 the reference level)
age_info$age_group <- ifelse(age_info$age == "0-79", "under 80", "over 80")
age_info$age_group <- factor(age_info$age_group, levels = c("under 80", "over
80"))

# Verify the structure
str(age_info)
```

```
## 'data.frame':    427 obs. of  5 variables:
##  $ patient_id: chr  "TCGA-3Z-A93Z" "TCGA-6D-AA2E" "TCGA-A3-3308" "TCGA-A3-
3311" ...
##  $ sample_id : chr  "TCGA.3Z.A93Z.01A.11R.A37O.07" "TCGA.6D.AA2E.01A.11R.A
37O.07" "TCGA.A3.3308.01A.02R.1325.07" "TCGA.A3.3311.01A.02R.1325.07" ...
```

```
##  $ AGE       : int  69 68 77 57 59 57 67 70 52 51 ...
##  $ age       : chr  "0-79" "0-79" "0-79" "0-79" ...
##  $ age_group : Factor w/ 2 levels "under 80","over 80": 1 1 1 1 1 1 1 1 1
1 ...
```

```
head(age_info)
```

```
##     patient_id                    sample_id AGE  age age_group
## 1 TCGA-3Z-A93Z TCGA.3Z.A93Z.01A.11R.A37O.07  69 0-79  under 80
## 2 TCGA-6D-AA2E TCGA.6D.AA2E.01A.11R.A37O.07  68 0-79  under 80
## 3 TCGA-A3-3308 TCGA.A3.3308.01A.02R.1325.07  77 0-79  under 80
## 4 TCGA-A3-3311 TCGA.A3.3311.01A.02R.1325.07  57 0-79  under 80
## 5 TCGA-A3-3313 TCGA.A3.3313.01A.02R.1325.07  59 0-79  under 80
## 6 TCGA-A3-3316 TCGA.A3.3316.01A.01R.0864.07  57 0-79  under 80
```

```
table(age_info$age_group)
```

```
##
## under 80  over 80
##      405       22
```

```
# Recode age (make under80 the reference level)
age_info$age_group <- ifelse(age_info$age == "0-79", "under 80", "over 80")
age_info$age_group <- factor(age_info$age_group, levels = c("under 80", "over
80"))

# Subset raw count matrix to matched samples
counts_sub <- mapped_count_matrix[, age_info$sample_id]
# Check dimensions
cat("Counts subset:", dim(counts_sub), "\n")
```

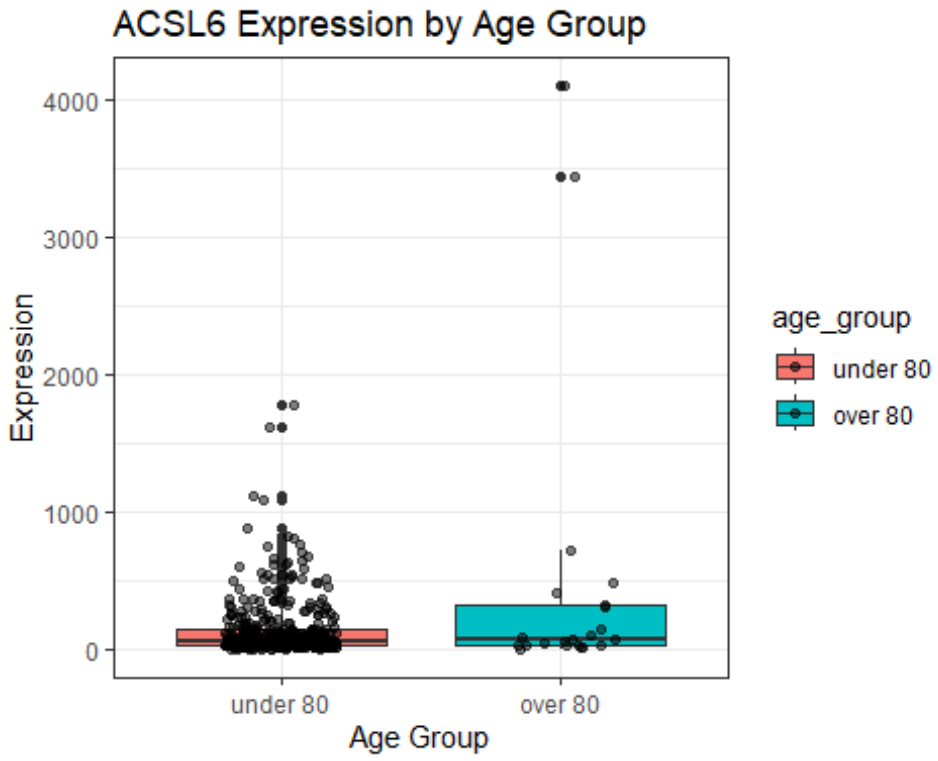
```
## Counts subset: 36585 427
```

```
# Filter: Keep genes with ≥10 counts in ≥20% of samples
min_samples <- 0.2 * ncol(counts_sub)   # ~85 samples

keep_genes <- rowSums(counts_sub >= 10) >= min_samples
filtered_counts <- counts_sub[keep_genes, ]

cat("Filtered count matrix dimensions:", dim(filtered_counts), "\n")
```

```
## Filtered count matrix dimensions: 19209 427
```

```
# Create DESeqDataset
dds <- DESeqDataSetFromMatrix(
    countData = filtered_counts,
    colData = data.frame(age_group = age_info$age_group,
                         row.names = age_info$sample_id),
    design = ~ age_group
)
```

```r
# Run DESeq2
dds <- DESeq(dds)

res <- results(dds, contrast = c("age_group", "over 80", "under 80"))

# Super simple summary
if("ACSL6" %in% rownames(res)) {
  cat("ACSL6 Results:\n")
  cat("Log2 Fold Change:", round(res["ACSL6", "log2FoldChange"], 3), "\n")
  cat("P-value:", formatC(res["ACSL6", "pvalue"], format = "e", digits = 2),
"\n")
  cat("Adjusted P-value:", formatC(res["ACSL6", "padj"], format = "e", digits
= 2), "\n")
}

## ACSL6 Results:
## Log2 Fold Change: 1.751
## P-value: 3.33e-07
## Adjusted P-value: 3.45e-04

if("ACSL6" %in% rownames(res)) {
  ACSL6_res <- res["ACSL6", ]
  print(ACSL6_res)
} else {
  print("ACSL6 not found in DESeq2 results — check symbol mapping.")
}

## log2 fold change (MLE): age_group over 80 vs under 80
## Wald test p-value: age group over.80 vs under.80
## DataFrame with 1 row and 6 columns
##        baseMean log2FoldChange     lfcSE      stat      pvalue        padj
##       <numeric>      <numeric> <numeric> <numeric>   <numeric>   <numeric>
## ACSL6   161.216        1.75086  0.343068   5.10353 3.33375e-07 0.000345475

# Get ACSL6 expression data
normalized_counts <- counts(dds, normalized = TRUE)
ACSL6_data <- data.frame(
  expression = normalized_counts["ACSL6", ],
  age_group = dds$age_group
)

# Simple boxplot
ggplot(ACSL6_data, aes(x = age_group, y = expression, fill = age_group)) +
  geom_boxplot() +
  geom_jitter(width = 0.2, alpha = 0.5) +
  labs(title = "ACSL6 Expression by Age Group",
       x = "Age Group",
       y = "Expression") +
  theme_bw()
```



Investigation 5

```
# Filter out X and Y Chromosome Genes

# Get chromosome information for the current gene set
# Note: Using Gene Symbols as keytype for a simpler mapping
gene_chr_map <- AnnotationDbi::select(
  org.Hs.eg.db,
  keys = rownames(mapped_count_matrix),
  columns = c("CHR"),
  keytype = "SYMBOL"
)

## Warning in .deprecatedColsMessage(): Accessing gene location information v
ia 'CHR','CHRLOC','CHRLOCEND' is
##   deprecated. Please use a range based accessor like genes(), or select()
with
##   columns values like TXCHROM and TXSTART on a TxDb or OrganismDb object
##   instead.

## 'select()' returned 1:many mapping between keys and columns

# Identify Autosomal Genes (Chromosome 1-22)
# Exclude X, Y, MT (mitochondrial), and any unassigned/unknown chromosomes (N
A)
# Autosomal chromosomes are '1' through '22'.
# We must first filter out NAs from the mapping step.
gene_chr_map <- gene_chr_map %>% filter(!is.na(CHR))
```

```r
# Filter for only autosomes (1-22)
autosomal_gene_map <- gene_chr_map %>%
  filter(CHR %in% as.character(1:22)) %>%
  distinct(SYMBOL) # Keep only the unique autosomal symbols

# Filter the count matrix to include only autosomal genes
autosomal_count_matrix <- mapped_count_matrix[autosomal_gene_map$SYMBOL, ]

cat("Number of genes removed (X, Y, MT, unassigned):",
    nrow(mapped_count_matrix) - nrow(autosomal_count_matrix), "\n")
```

```
## Number of genes removed (X, Y, MT, unassigned): 1832
```

```r
cat("Count matrix dimensions (Autosomal Genes x Samples):",
    dim(autosomal_count_matrix), "\n")
```

```
## Count matrix dimensions (Autosomal Genes x Samples): 34753 427
```

```r
# Reassign the count matrix variable to be used in the next step
count_data_to_filter <- autosomal_count_matrix

# Check that sex-chromosome linked genes were removed
# Get chromosome mapping for the filtered (autosomal) count matrix
filtered_chr_map <- AnnotationDbi::select(
  org.Hs.eg.db,
  keys = rownames(count_data_to_filter),
  columns = c("CHR"),
  keytype = "SYMBOL"
)

# Count how many genes map to X or Y
sum(filtered_chr_map$CHR %in% c("X", "Y"))
```

```
## [1] 0
```

```r
# List any genes on X or Y (should be empty)
filtered_chr_map %>% filter(CHR %in% c("X", "Y"))
```

```
## [1] SYMBOL CHR
## <0 rows> (or 0-length row.names)
```

```r
# if output is empty then no x/y genes remain

# Extract sample IDs from your count matrix column names
# Use the autosomal count matrix bc filtering sex chromosomes
sample_ids <- colnames(count_data_to_filter)

# Clean up sample IDs to match patient IDs
patient_ids <- gsub("\\.", "-", substr(sample_ids, 1, 12))

# Create sex_info by matching with clinical data
```

```r
sex_info <- data.frame(
  sample_id = sample_ids,
  patient_id = patient_ids,
  stringsAsFactors = FALSE
)

# Merge with clinical data to get sex information
sex_info <- merge(
  sex_info,
  data_clinical_patient[, c("PATIENT_ID", "SEX")],  # Adjust "SEX" if column
name differs
  by.x = "patient_id",
  by.y = "PATIENT_ID",
  all.x = TRUE
)

# Rename the SEX column to lowercase for consistency
names(sex_info)[names(sex_info) == "SEX"] <- "sex"

# Filter out any missing or empty sex values
sex_info <- sex_info %>%
  filter(!is.na(sex) & sex != "")

# Set sex as a factor with Male as reference level
sex_info$sex <- factor(sex_info$sex, levels = c("Male", "Female"))

# Verify the structure
str(sex_info)
```

```
## 'data.frame':    427 obs. of  3 variables:
##  $ patient_id: chr  "TCGA-3Z-A93Z" "TCGA-6D-AA2E" "TCGA-A3-3308" "TCGA-A3-
3311" ...
##  $ sample_id : chr  "TCGA.3Z.A93Z.01A.11R.A37O.07" "TCGA.6D.AA2E.01A.11R.A
37O.07" "TCGA.A3.3308.01A.02R.1325.07" "TCGA.A3.3311.01A.02R.1325.07" ...
##  $ sex       : Factor w/ 2 levels "Male","Female": 1 2 2 1 1 1 1 1 2 1 ...
```

```r
head(sex_info)
```

```
##     patient_id                    sample_id    sex
## 1 TCGA-3Z-A93Z TCGA.3Z.A93Z.01A.11R.A37O.07   Male
## 2 TCGA-6D-AA2E TCGA.6D.AA2E.01A.11R.A37O.07 Female
## 3 TCGA-A3-3308 TCGA.A3.3308.01A.02R.1325.07 Female
## 4 TCGA-A3-3311 TCGA.A3.3311.01A.02R.1325.07   Male
## 5 TCGA-A3-3313 TCGA.A3.3313.01A.02R.1325.07   Male
## 6 TCGA-A3-3316 TCGA.A3.3316.01A.01R.0864.07   Male
```

```r
table(sex_info$sex)
```

```
##
##   Male Female
##    274    153

# Check that sample_ids match between sex_info and count matrix
cat("Samples in sex_info:", nrow(sex_info), "\n")

## Samples in sex_info: 427

cat("Samples in count matrix:", ncol(count_data_to_filter), "\n")

## Samples in count matrix: 427

# Subset the mapped count matrix to samples that have sex information
counts_sub_sex <- count_data_to_filter[, sex_info$sample_id]

# Check dimensions
cat("Counts subset for sex comparison:", dim(counts_sub_sex), "\n")

## Counts subset for sex comparison: 34753 427

# Apply low count filter
min_samples_sex <- 0.2 * ncol(counts_sub_sex)

keep_genes_sex <- rowSums(counts_sub_sex >= 10) >= min_samples_sex

filtered_counts_sex <- counts_sub_sex[keep_genes_sex, ]
sex_info$sex <- factor(sex_info$sex, levels = c("Male", "Female"))

dds_sex <- DESeqDataSetFromMatrix(
    countData = filtered_counts_sex,
    colData = data.frame(
        sex = sex_info$sex,
        row.names = sex_info$sample_id
    ),
    design = ~ sex
)

# Pre-filter low counts
# Keep genes with at least 10 reads across all samples
dds_sex <- dds_sex[rowSums(counts(dds_sex)) >= 10, ]

# Run DESeq2
dds_sex <- DESeq(dds_sex)

colnames(colData(dds_sex))

## [1] "sex"         "sizeFactor"  "replaceable"

# Get results
res_sex <- results(dds_sex, contrast = c("sex", "Female", "Male"))
# This gives log2FC = Female vs Male
```

```r
# Sort by adjusted p-value
res_sex <- res_sex[order(res_sex$padj), ]

# View top genes
head(res_sex, 10)
```

```
## log2 fold change (MLE): sex Female vs Male
## Wald test p-value: sex Female vs Male
## DataFrame with 10 rows and 6 columns
##            baseMean log2FoldChange     lfcSE      stat      pvalue
padj
##           <numeric>      <numeric> <numeric> <numeric>   <numeric>   <nume
ric>
## LINC01554  373.3506       -3.16423  0.227646  -13.8998 6.35296e-44 1.17733
e-39
## TRIM63      99.5476        3.10457  0.234425   13.2434 4.92833e-40 4.56659
e-36
## KLK1       609.5479       -4.17056  0.337833  -12.3451 5.17961e-35 3.19962
e-31
## CCDC146   4499.3134       -2.09088  0.170769  -12.2439 1.81165e-34 8.39338
e-31
## GPR15LG    994.3925       -3.73866  0.305952  -12.2198 2.43808e-34 9.03650
e-31
## APOB      1796.8057       -2.73989  0.234659  -11.6760 1.68999e-31 4.47413
e-28
## TRAV30     255.5514       -1.80704  0.154612  -11.6876 1.47538e-31 4.47413
e-28
## TUNAR      101.5332       -3.56535  0.306257  -11.6417 2.52854e-31 5.85737
e-28
## TUBA3D     594.3655       -2.92990  0.252994  -11.5809 5.14929e-31 1.06030
e-27
## GSG1L2     106.9267       -5.04790  0.452802  -11.1481 7.31225e-29 1.35511
e-25
```

```r
# Optional: convert to a data frame for easier manipulation
res_df <- as.data.frame(res_sex)
res_df$gene <- rownames(res_df)

# Filter for significantly DE genes (FDR < 0.05)
sig_res <- res_df[!is.na(res_df$padj) & res_df$padj < 0.05, ]

# Order by absolute log2 fold change, largest first
sig_res_absFC <- sig_res[order(abs(sig_res$log2FoldChange), decreasing = TRUE
), ]

#console shows top most significant genes
#df shows most differentially expressed genes (by log2fc)
# View top 10 genes with largest absolute log2FC
head(sig_res_absFC, 10)
```

```
##              baseMean log2FoldChange     lfcSE       stat       pvalue
## GSG1L2     106.92669      -5.047899 0.4528021 -11.148135 7.312255e-29
## KLK1       609.54788      -4.170563 0.3378326 -12.345059 5.179613e-35
## GPR15LG    994.39255      -3.738661 0.3059521 -12.219758 2.438080e-34
## TUNAR      101.53319      -3.565355 0.3062566 -11.641723 2.528544e-31
## LINC01554  373.35063      -3.164228 0.2276458 -13.899788 6.352957e-44
## TRIM63      99.54757       3.104573 0.2344246  13.243375 4.928331e-40
## TUBA3D     594.36554      -2.929898 0.2529936 -11.580916 5.149287e-31
## TUBA3E      66.90267      -2.862682 0.3220466  -8.889031 6.164402e-19
## LINC03020   17.03923      -2.856735 0.2753187 -10.376104 3.185105e-25
## APOB      1796.80567      -2.739885 0.2346589 -11.676034 1.689991e-31
##                    padj      gene
## GSG1L2    1.355107e-25    GSG1L2
## KLK1      3.199620e-31      KLK1
## GPR15LG   9.036500e-31   GPR15LG
## TUNAR     5.857372e-28     TUNAR
## LINC01554 1.177330e-39 LINC01554
## TRIM63    4.566591e-36    TRIM63
## TUBA3D    1.060295e-27    TUBA3D
## TUBA3E    4.393796e-16    TUBA3E
## LINC03020 3.689147e-22 LINC03020
## APOB      4.474131e-28      APOB
```

```r
# Vector of genes to inspect
genes_of_interest <- c("CCDC146", "UGT1A6")

# Subset DESeq2 results for these genes
res_subset <- res_sex[rownames(res_sex) %in% genes_of_interest, ]

# If you want to see it as a clean data frame
res_subset_df <- as.data.frame(res_subset)
res_subset_df$gene <- rownames(res_subset_df)  # add gene column for clarity

# Print results
print(res_subset_df)
```

```
##         baseMean log2FoldChange     lfcSE       stat       pvalue
padj
## CCDC146 4499.313     -2.0908806 0.1707695 -12.243878 1.811652e-34 8.393385
e-31
## UGT1A6  1008.617      0.9007972 0.1713999   5.255528 1.476001e-07 1.254736
e-05
##            gene
## CCDC146 CCDC146
## UGT1A6   UGT1A6
```

#Investigation 6

```r
library(ggplot2)
library(pheatmap)
```

```r
library(biomaRt)
library(dplyr)

#Clean Ensembl IDs and map to gene symbols

# Remove version numbers from Ensembl IDs
clean_ids <- sub("\\..*", "", rownames(log_norm_counts))
rownames(log_norm_counts) <- clean_ids
rownames(z_log) <- clean_ids

# Get gene symbols from Ensembl
mart <- useMart("ensembl", dataset = "hsapiens_gene_ensembl")
mapping <- getBM(
  attributes = c("ensembl_gene_id", "hgnc_symbol"),
  filters = "ensembl_gene_id",
  values = clean_ids,
  mart = mart
)

# Convert to gene symbols
expr_df <- as.data.frame(log_norm_counts) %>%
  mutate(ensembl_gene_id = rownames(log_norm_counts)) %>%
  left_join(mapping, by = "ensembl_gene_id") %>%
  filter(hgnc_symbol != "") %>%
  distinct(hgnc_symbol, .keep_all = TRUE)

rownames(expr_df) <- expr_df$hgnc_symbol
expr_symbol <- as.matrix(expr_df[, -c(ncol(expr_df)-1, ncol(expr_df))])

# Same for z-scores
z_df <- as.data.frame(z_log) %>%
  mutate(ensembl_gene_id = rownames(z_log)) %>%
  left_join(mapping, by = "ensembl_gene_id") %>%
  filter(hgnc_symbol != "") %>%
  distinct(hgnc_symbol, .keep_all = TRUE)

rownames(z_df) <- z_df$hgnc_symbol
z_scores_symbol <- as.matrix(z_df[, -c(ncol(z_df)-1, ncol(z_df))])

cat("Genes with symbols:", nrow(expr_symbol), "\n\n")

## Genes with symbols: 41248

# Subset to Samples(Patients) with Stage Info

# Extract patient IDs and match to clinical data
rnaseq_patient_ids <- substr(gsub("\\.", "-", colnames(expr_symbol)), 1, 12)

stage_info <- data.frame(
  sample_id = colnames(expr_symbol),
```

```r
  patient_id = rnaseq_patient_ids
) %>%
  left_join(
    data_clinical_patient %>%
      dplyr::select(PATIENT_ID, AJCC_PATHOLOGIC_TUMOR_STAGE) %>%
      rename(patient_id = PATIENT_ID, stage = AJCC_PATHOLOGIC_TUMOR_STAGE),
    by = "patient_id"
  ) %>%
  filter(!is.na(stage) & stage != "")

# Subset expression matrices
expr_staged <- expr_symbol[, stage_info$sample_id]
z_scores_staged <- z_scores_symbol[, stage_info$sample_id]

cat("Samples with stage:", nrow(stage_info), "\n")
```

```
## Samples with stage: 427
```

```r
print(table(stage_info$stage))
```

```
##
##   STAGE I  STAGE II STAGE III  STAGE IV
##       211        53        95        68
```

```r
cat("\n")

#Select top 500 Most Variable genes

# Calculate variance for each gene
gene_variance <- apply(expr_staged, 1, var)

# Select top 500
top500_genes <- names(sort(gene_variance, decreasing = TRUE)[1:500])

cat("Top 500 genes selected\n")
```

```
## Top 500 genes selected
```

```r
cat("Variance range:", round(range(gene_variance[top500_genes]), 2), "\n\n")
```

```
## Variance range: 7.22 39.46
```

```r
#This shows how expressed each gene is, and take the top 500 genes that have
the largest variance in expression.
#Each patient expresses a certain amount of each gene,
#so the highest variance is intersting as something might be happening, Could
be indicative.

#ANOVA Test by stage

# Test if genes differ across stages
```

```r
anova_results <- data.frame(
  gene = top500_genes,
  p_value = sapply(top500_genes, function(g) {
    fit <- aov(expr_staged[g, ] ~ factor(stage_info$stage))
    summary(fit)[[1]]["Pr(>F)"][1, 1]
  })
)

#Multiple testing correction

anova_results$p_adj <- p.adjust(anova_results$p_value, method = "BH")
anova_results$significant <- anova_results$p_adj < 0.05

cat("Significant genes (FDR < 0.05):", sum(anova_results$significant), "\n")
```

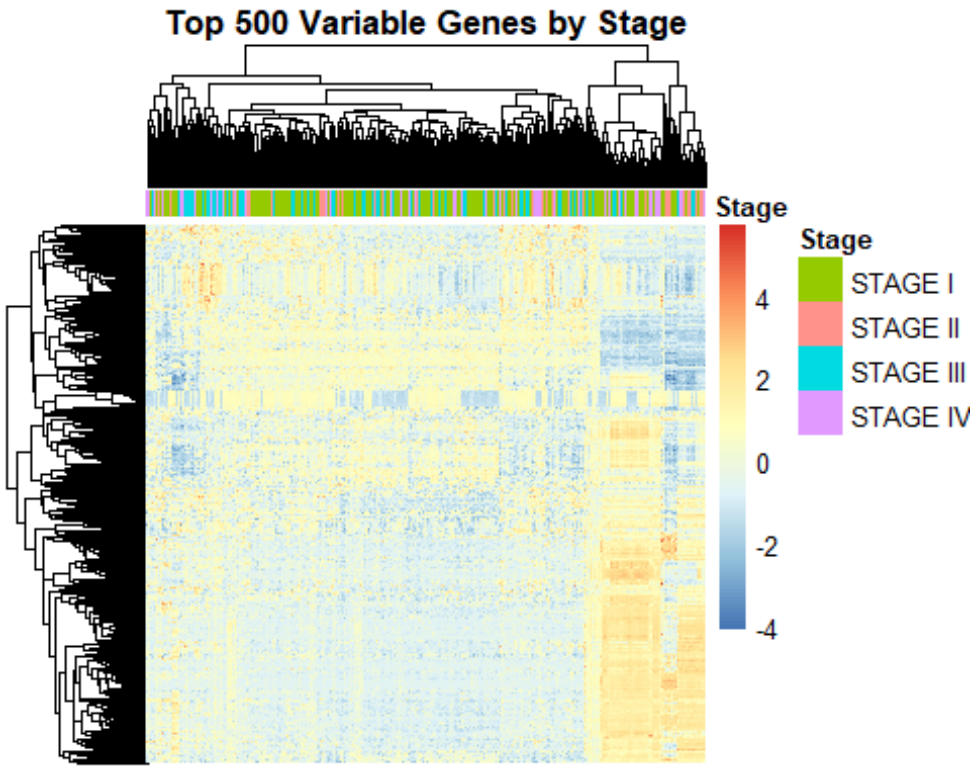
```
## Significant genes (FDR < 0.05): 85
```

```r
print(anova_results %>% arrange(p_adj) %>% head(10))
```

```
##            gene      p_value        p_adj significant
## SAA1       SAA1 2.016938e-09 1.008469e-06        TRUE
## PRAME     PRAME 3.236906e-08 8.092266e-06        TRUE
## DNER       DNER 2.442087e-07 2.866516e-05        TRUE
## PAEP       PAEP 2.539381e-07 2.866516e-05        TRUE
## PITX2     PITX2 2.866516e-07 2.866516e-05        TRUE
## REG3G     REG3G 3.712902e-07 3.094085e-05        TRUE
## PITX1     PITX1 7.129368e-07 5.092406e-05        TRUE
## MAT1A     MAT1A 1.287409e-06 7.762228e-05        TRUE
## C1QL1     C1QL1 1.397201e-06 7.762228e-05        TRUE
## PPP1R1A PPP1R1A 2.592792e-06 1.296396e-04        TRUE
```

```r
cat("\n")

#Heatmap Vizualization
#Out of the 500 most variable, only 85 are significant -> therefore, should o
nly see small clustering....

top500_matrix <- z_scores_staged[top500_genes, stage_info$sample_id]

annotation <- data.frame(
  Stage = factor(stage_info$stage),
  row.names = stage_info$sample_id
)

pheatmap(
  top500_matrix,
  annotation_col = annotation,
  show_rownames = FALSE,
  show_colnames = FALSE,
```

```
  main = "Top 500 Variable Genes by Stage"
)
```




```
#Identify the oncogenes

oncogenes <- c("MYC","KRAS","NRAS","HRAS","EGFR","ERBB2","PIK3CA",
               "BRAF","AKT1","AKT2","MDM2","CCND1","CDK4","CDK6",
               "MET","RET","ALK","ROS1","FGFR1","FGFR2")

oncogenes_in_data <- intersect(rownames(expr_staged), oncogenes)

cat("Oncogenes found:", length(oncogenes_in_data), "\n")

## Oncogenes found: 20

cat(paste(oncogenes_in_data, collapse = ", "), "\n\n")

## ROS1, FGFR2, FGFR1, AKT2, CDK6, MET, CCND1, PIK3CA, KRAS, CDK4, MDM2, MYC,
ERBB2, AKT1, EGFR, BRAF, RET, ALK, HRAS, NRAS

#Oncogene stage correlation
#Spearman correlation is a monotonic relationship between 2 variables
#In this case, we are looking at does expression go up or down with stage

if (length(oncogenes_in_data) > 0) {

  # Convert stage to numeric (I=1, II=2, III=3, IV=4)
```

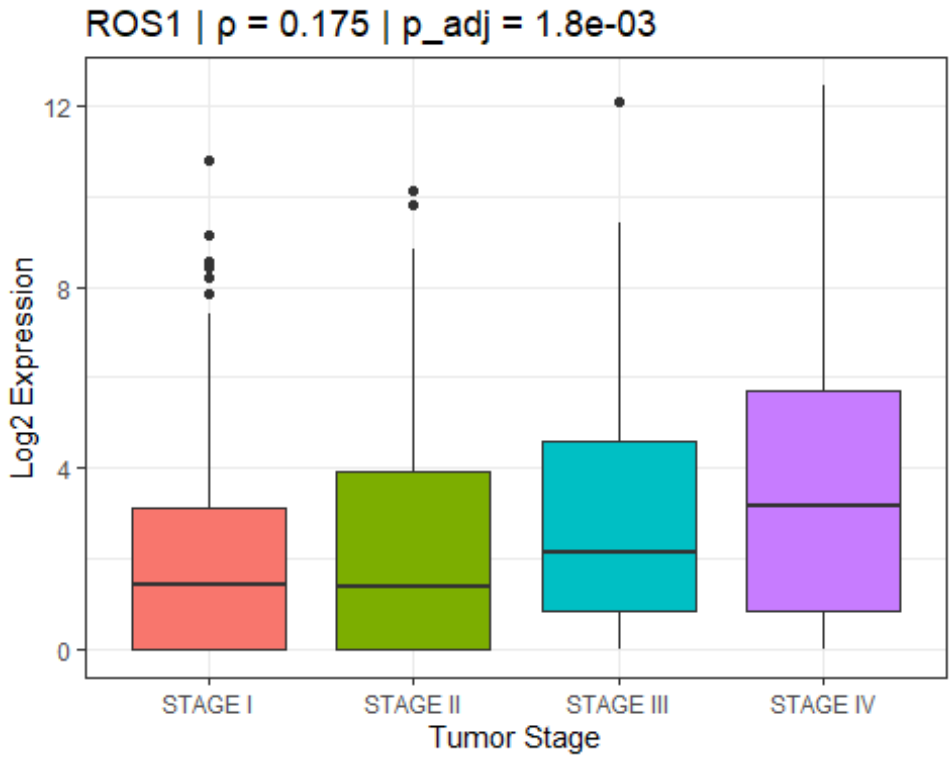
```r
  stage_numeric <- as.numeric(factor(
    stage_info$stage,
    levels = c("STAGE I", "STAGE II", "STAGE III", "STAGE IV")
  ))

  # Test correlation for each oncogene
  oncogene_results <- data.frame(
    gene = oncogenes_in_data,
    correlation = sapply(oncogenes_in_data, function(g) {
      cor.test(stage_numeric, expr_staged[g, ], method = "spearman")$estimate
    }),
    p_value = sapply(oncogenes_in_data, function(g) {
      cor.test(stage_numeric, expr_staged[g, ], method = "spearman")$p.value
    })
  )

  oncogene_results$p_adj <- p.adjust(oncogene_results$p_value, method = "BH")
  oncogene_results$significant <- oncogene_results$p_adj < 0.05
  oncogene_results$direction <- ifelse(oncogene_results$correlation > 0, "UP"
, "DOWN")

  cat("Oncogene correlation results:\n")
  print(oncogene_results %>% arrange(p_adj))
  cat("\n")
}
```

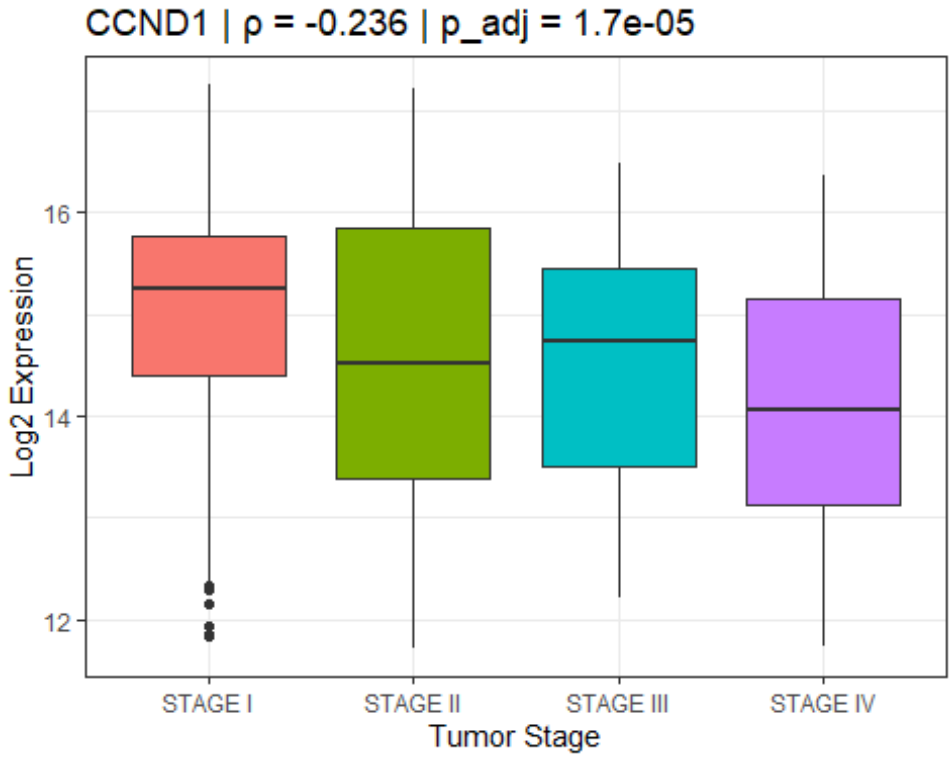
```
## Oncogene correlation results:
##              gene  correlation      p_value        p_adj significant direc
tion
## CCND1.rho   CCND1 -0.235514156 8.575424e-07 1.715085e-05        TRUE
DOWN
## AKT1.rho     AKT1 -0.193548043 5.676268e-05 5.676268e-04        TRUE
DOWN
## ROS1.rho     ROS1  0.175492276 2.683737e-04 1.789158e-03        TRUE
UP
## BRAF.rho     BRAF -0.119345611 1.359646e-02 6.798229e-02       FALSE
DOWN
## PIK3CA.rho PIK3CA -0.111509046 2.118575e-02 7.867483e-02       FALSE
DOWN
## HRAS.rho     HRAS  0.109531667 2.360245e-02 7.867483e-02       FALSE
UP
## FGFR1.rho   FGFR1 -0.063489054 1.903948e-01 3.539717e-01       FALSE
DOWN
## AKT2.rho     AKT2  0.062268784 1.990736e-01 3.539717e-01       FALSE
UP
## CDK6.rho     CDK6  0.069770572 1.500752e-01 3.539717e-01       FALSE
UP
## ERBB2.rho   ERBB2 -0.060470496 2.123830e-01 3.539717e-01       FALSE
DOWN
## EGFR.rho     EGFR -0.063001576 1.938279e-01 3.539717e-01       FALSE
```

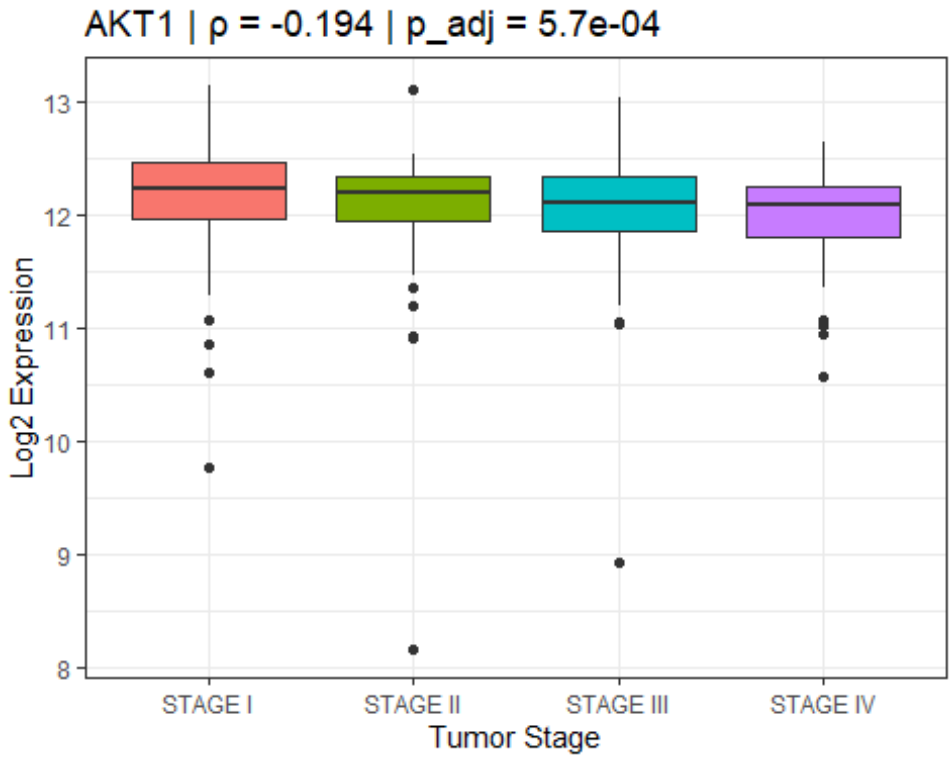
```
DOWN
## NRAS.rho     NRAS -0.062638998 1.964106e-01 3.539717e-01       FALSE
DOWN
## RET.rho       RET  0.055709833 2.506778e-01 3.856582e-01       FALSE
UP
## MDM2.rho     MDM2 -0.046543452 3.373202e-01 4.818860e-01       FALSE
DOWN
## ALK.rho       ALK -0.034817199 4.730222e-01 6.306963e-01       FALSE
DOWN
## MET.rho       MET  0.031961211 5.101025e-01 6.376281e-01       FALSE
UP
## KRAS.rho     KRAS  0.023603807 6.266931e-01 7.372860e-01       FALSE
UP
## FGFR2.rho   FGFR2  0.017860223 7.128652e-01 7.920725e-01       FALSE
UP
## CDK4.rho     CDK4 -0.013941483 7.739163e-01 8.146488e-01       FALSE
DOWN
## MYC.rho       MYC  0.001207987 9.801438e-01 9.801438e-01       FALSE
UP
```

```
#Plot significant outcomes

if (exists("oncogene_results")) {
  sig_oncogenes <- oncogene_results %>%
    filter(significant) %>%
    pull(gene)

  if (length(sig_oncogenes) > 0) {
    for (gene in sig_oncogenes) {
      corr <- oncogene_results$correlation[oncogene_results$gene == gene]
      p_adj <- oncogene_results$p_adj[oncogene_results$gene == gene]

      plot_data <- data.frame(
        expression = expr_staged[gene, ],
        stage = factor(stage_info$stage)
      )

      p <- ggplot(plot_data, aes(stage, expression, fill = stage)) +
        geom_boxplot() +
        labs(
          title = paste(gene, "| ρ =", round(corr, 3),
                        "| p_adj =", format(p_adj, scientific = TRUE, digits =
2)),
          x = "Tumor Stage",
          y = "Log2 Expression"
        ) +
        theme_bw() +
        theme(legend.position = "none")
```

```
      print(p)
    }
  } else {
    cat("No significant oncogenes to plot\n")
  }
}
```


ROS1 | ρ = 0.175 | p_adj = 1.8e-03
Log2 Expression
12
8
4
0
STAGE I
STAGE II
STAGE III
STAGE IV
Tumor Stage


CCND1 | ρ = -0.236 | p_adj = 1.7e-05
Log2 Expression
16
14
12
STAGE I
STAGE II
STAGE III
STAGE IV
Tumor Stage


AKT1 | ρ = -0.194 | p_adj = 5.7e-04
Log2 Expression
13
12
11
10
9
8
STAGE I
STAGE II
STAGE III
STAGE IV
Tumor Stage